

Modern sequence modeling is defined by a frustrating trade-off: Transformers offer unmatched expressive power by keeping every past token in memory, but their training costs scale quadratically with sequence length and their inference crawls under the weight of growing key-value (KV) caches. Recurrent models and classical State Space Models (SSMs) scale linearly during training and generate tokens in constant time per step, yet their performance degrades because their static hidden states compress historical context poorly.

Mamba resolves this tension by introducing **selective state space models**. By parameterizing the underlying state space dynamics directly as functions of the current input token, Mamba selectively filters out irrelevant noise, retains critical context indefinitely, and matches or exceeds the pretraining perplexity and downstream accuracy of modern Transformer architectures while generating text four to five times faster.

## The Context Compression Dilemma

At its heart, sequence modeling is context compression. Transformers refuse to compress context at all: every token attends to all previous tokens, which explains their strength on tasks requiring associative recall, like induction heads. But this lack of compression means an expanding KV cache during autoregressive generation, which causes linear inference slowdown and quadratic memory expansion.

To achieve linear scaling, prior structured SSMs such as S4 mapped continuous differential equations into discrete linear recurrences:

$$h_t = \bar{A}h_{t-1} + \bar{B}x_t, \quad y_t = Ch_t$$

Here, the continuous parameters  $(\Delta, A, B)$  are transformed into discrete matrices  $(\bar{A}, \bar{B})$  via discretization rules like Zero-Order Hold (ZOH):

$$\bar{A} = \exp(\Delta A), \quad \bar{B} = (\Delta A)^{-1}(\exp(\Delta A) - I) \cdot \Delta B$$

To make this recurrence efficient on parallel hardware, prior SSMs enforced **Linear Time Invariance (LTI)**. Because the transition matrices  $(\bar{A}, \bar{B}, C)$  stayed constant for every single token across time, the entire sequence recurrence could be unrolled and computed as a single global convolution using Fast Fourier Transforms.

However, LTI is a double-edged sword. Because the transition weights never change with token content, the model cannot alter its hidden memory based on what it reads. On synthetic benchmarks like Selective Copying—where the model must memorize colored tokens and skip white spacer tokens—fixed convolution kernels fail because the spacing between target tokens varies dynamically.

## Making Dynamics Selective: The S6 Mechanism

Mamba breaks the LTI constraint by making the SSM parameters input-dependent. Instead of treating  $B$ ,  $C$ , and the step size  $\Delta$  as static parameters learned across the whole dataset, Mamba calculates them on the fly from the current input token  $x$ :

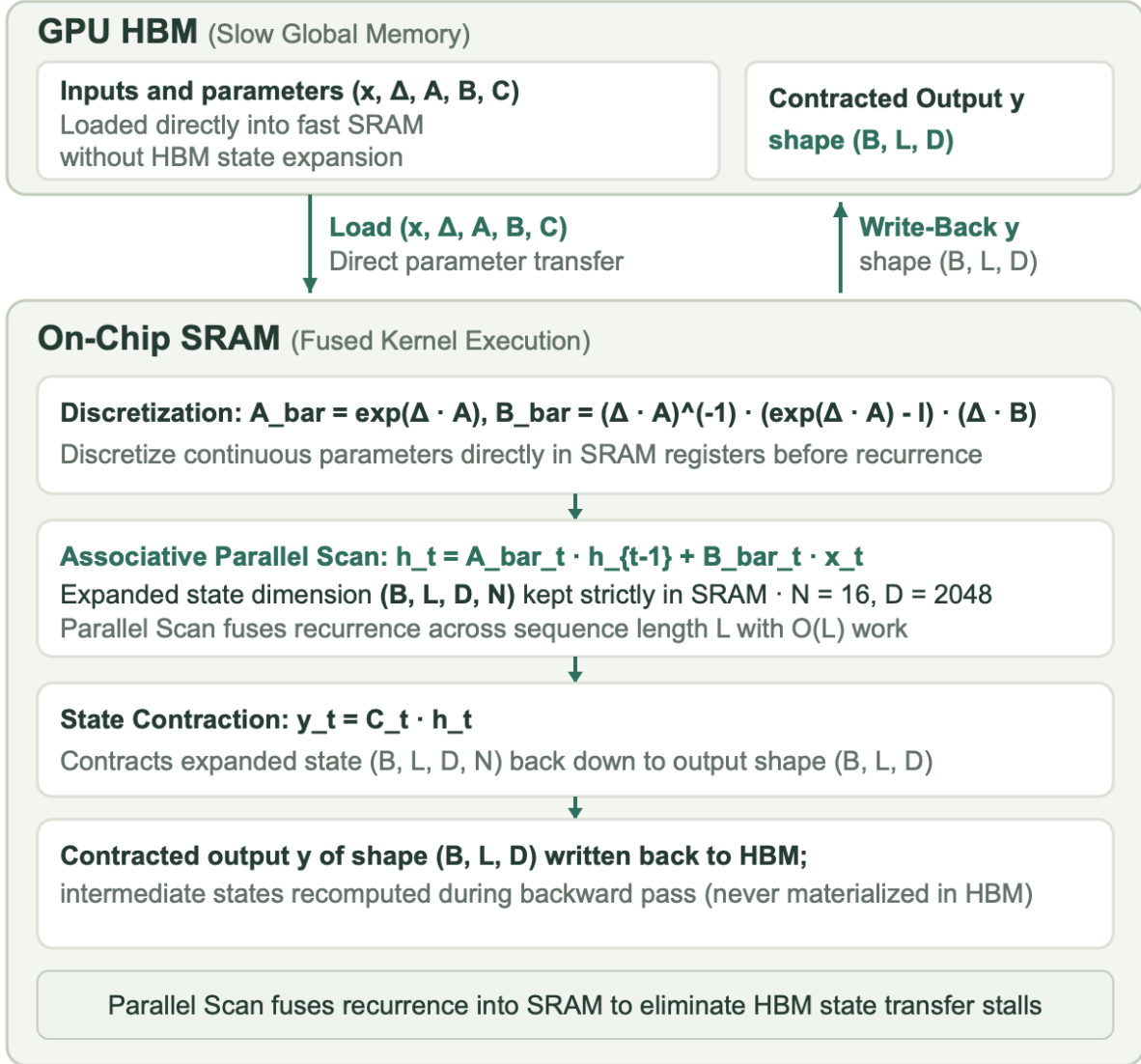
$$B = \text{Linear}_N(x), \quad C = \text{Linear}_N(x), \quad \Delta = \text{softplus}(\text{Parameter} + \text{Broadcast}_D(\text{Linear}_1(x)))$$

This algorithm, termed S6 (Selective SSM), equips the model with dynamic gating. The continuous discretization step size  $\Delta$  plays a crucial role: 1. When  $\Delta$  is large,  $\bar{A} = \exp(\Delta A)$  shrinks toward zero, effectively resetting the hidden state  $h$ , while  $\bar{B}$  emphasizes the current token  $x$ . The model focuses intensely on the incoming content. 2. When  $\Delta$  is small,  $\bar{A}$  approaches the identity matrix, preserving the existing recurrent state  $h$ , while  $\bar{B}$  decays toward zero, filtering out the current token as transient noise.

## Overcoming Memory Bottlenecks with Hardware-Aware Scans

Making dynamics time-varying comes with a heavy computational penalty: it forfeits equivalence to global convolutions. Training can no longer rely on parallel FFTs.

A naive recurrent scan across batch size  $B$ , sequence length  $L$ , model dimension  $D$ , and state expansion factor  $N$  requires materializing a state tensor of shape  $(B, L, D, N)$ . For standard configurations like  $N = 16$  and sequence lengths of thousands of tokens, reading and writing this expanded state between GPU High-Bandwidth Memory (HBM) and processor registers chokes the memory bus, stalling execution.



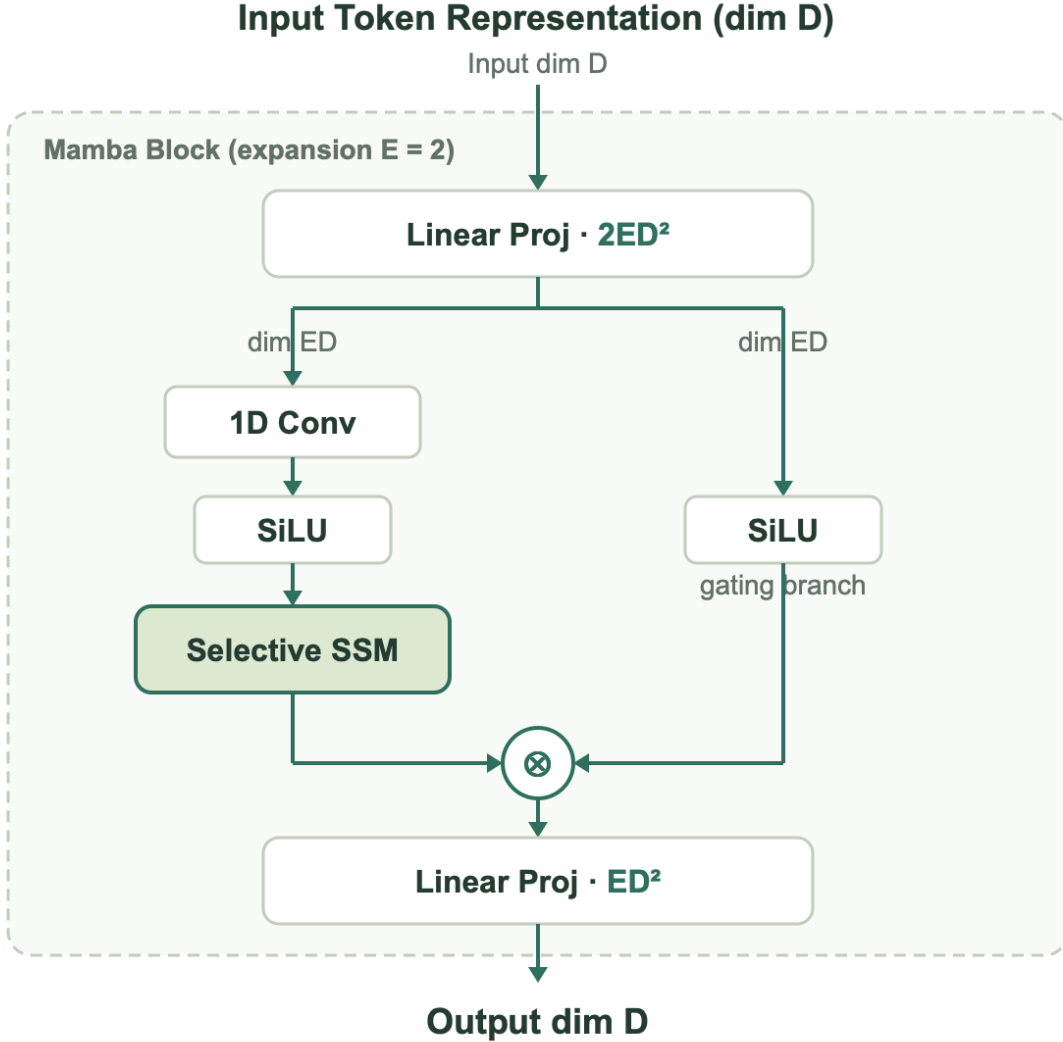
Mamba avoids memory bandwidth bottlenecks by loading compact parameters into fast SRAM, computing discretization and recurrence on-chip, and returning only contracted outputs to HBM.

Mamba solves this hardware bottleneck through a fused kernel design: 1. **Kernel Fusion in SRAM:** Instead of creating intermediate discretized matrices  $(\bar{A}, \bar{B})$  in slow HBM, Mamba loads the compact vectors  $(\Delta, A, B, C)$  directly into fast on-chip SRAM. Discretization, recurrence, and output contraction ( $y = Ch$ ) execute entirely inside SRAM, writing only the final output  $(B, L, D)$  back to HBM. 2. **Associative Parallel Scan:** Recurrence along the time axis is parallelized using work-efficient parallel scan algorithms, converting the sequential loop into an  $O(L)$  parallel operation. 3. **Recomputation:** To preserve memory during

backward propagation, intermediate states are not stored in HBM; instead, they are cleanly recomputed in SRAM during the backward pass.

### The Simplified Homogeneous Architecture

Prior SSM models like H3 interleaved specialized recurrent blocks with standard Transformer Multi-Layer Perceptrons (MLPs). Mamba simplifies this design into a single repeating block.



The Mamba block expands the input dimension by factor  $E=2$  into dual branches, processes the primary branch through 1D convolution and the selective SSM core, and recombines via multiplicative gating.

Inspired by Gated Attention Units and SwiGLU MLPs, each Mamba block expands the input dimension  $D$  by a factor  $E = 2$ . The parameters are dominated by linear projections ( $3ED^2$  total:  $2ED^2$  on input expansion and  $ED^2$  on output contraction), while the inner SSM requires relatively few weights. The primary branch applies a short 1D convolution to capture local context before the selective SSM, while the parallel branch applies a SiLU activation to multiplicatively gate the recurrent output. Stacking two Mamba blocks preserves the exact  $12D^2$  parameter budget of a standard Transformer layer.

## Empirical Results Across Modalities

Across language modeling benchmarks, Mamba matches or exceeds modern Transformer recipes (such as Transformer++ / LLaMa) across scaling laws up to 3B parameters.

| Model        | Parameters  | Tokenizer | LAMBADA        | PIQA Acc $\uparrow$ | ARC-E Acc    | Average              |
|--------------|-------------|-----------|----------------|---------------------|--------------|----------------------|
|              |             |           | Acc $\uparrow$ |                     | $\uparrow$   | Zero-Shot $\uparrow$ |
| Pythia       | 160M        | NeoX      | 33.0%          | 61.4%               | 43.2%        | 40.6%                |
| <b>Mamba</b> | <b>130M</b> | NeoX      | <b>44.3%</b>   | <b>64.5%</b>        | <b>48.0%</b> | <b>44.7%</b>         |
| Pythia       | 410M        | NeoX      | 51.4%          | 66.9%               | 52.1%        | 48.2%                |
| <b>Mamba</b> | <b>370M</b> | NeoX      | <b>55.6%</b>   | <b>69.5%</b>        | <b>55.1%</b> | <b>50.0%</b>         |
| Pythia       | 1.4B        | NeoX      | 61.7%          | 71.0%               | 60.5%        | 55.2%                |
| RWKV         | 1.5B        | NeoX      | 56.4%          | 72.4%               | 60.5%        | 54.3%                |
| <b>Mamba</b> | <b>1.4B</b> | NeoX      | <b>64.9%</b>   | <b>74.2%</b>        | <b>65.5%</b> | <b>59.7%</b>         |
| Pythia       | 2.8B        | NeoX      | 64.7%          | 74.0%               | 64.1%        | 59.1%                |
| RWKV         | 3B          | NeoX      | 63.9%          | 73.7%               | 67.8%        | 59.6%                |
| <b>Mamba</b> | <b>2.8B</b> | NeoX      | <b>69.2%</b>   | <b>75.2%</b>        | <b>69.7%</b> | <b>63.3%</b>         |

On downstream evaluations, Mamba models consistently match or outperform open-source baselines twice their size. For example, Mamba-1.4B scores 59.7% average accuracy, surpassing both Pythia-1.4B (55.2%) and Pythia-2.8B (59.1%). During generation, because Mamba stores only a fixed-size recurrent state rather than an ever-growing KV cache, it achieves 4–5 $\times$  higher inference throughput than Transformers of similar size.

Beyond language, Mamba scales effectively to million-length contexts in DNA genomics and raw audio waveforms, solving synthetic tasks like induction heads across sequences exceeding 1M tokens.

## Limitations and Scope

Despite its advantages, selectivity brings inherent trade-offs. Moving away from LTI sacrifices convolution equivalence, requiring dedicated, hardware-specific GPU kernels that are complex to implement across different accelerator architectures.

Furthermore, there is a continuous-discrete trade-off. Classical LTI SSMs possess strong inductive biases for continuous-time signals like audio and video. While selectivity provides critical benefits on discrete symbolic tokens like text and DNA, the authors note that it can impede modeling performance on certain continuous perceptual signals where classical LTI SSMs excel. Within discrete domains, however, Mamba establishes that recurrent models can match Transformer quality while scaling linearly.